UFMG

Program Visualization

Compilers Laboratory

Fernando Magno Quintão Pereira
fernando@dcc.ufmg.br

The LLVM Optimizer

DCC

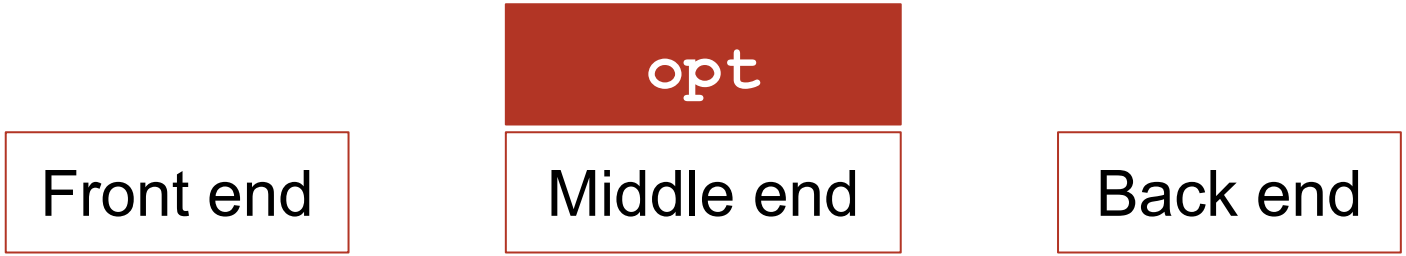
The LLVM Optimizer

DCC

`opt`

The LLVM Optimizer

DCC



The LLVM Optimizer

Command Line Tool

DCC

Command Line Tool

```
$> cd ~/Programs/llvm9/build/bin
```

Command Line Tool

DCC

```
$> cd ~/Programs/llvm9/build/bin
$> ~/Programs/llvm9/build/bin$ ls | grep opt
llvm-opt-fuzzer
llvm-opt-report
opt
```

Command Line Tool

```
$> opt --version
```

Command Line Tool

```
$> opt --version
LLVM (http://llvm.org/):
  LLVM version 11.0.0git
  DEBUG build with assertions.
  Default target: x86_64-unknown-linux-gnu
  Host CPU: sandybridge
```

Command Line Tool

```
$> opt --version
LLVM (http://llvm.org/):
  LLVM version 11.0.0git
  DEBUG build with assertions.
  Default target: x86_64-unknown-linux-gnu
  Host CPU: sandybridge
```

Using OPT

Using OPT

DCC

Visualization

Using OPT

Visualization

Analysis

Using OPT

Visualization

Analysis

Transformation

Using OPT

DCC

Visualization

Analysis

Transformation

Instrumentation

Using OPT

DCC

Visualization

Analysis

Transformation

Instrumentation

Optimization

Program Visualization

DCC

Visualization

Program Visualization

DCC

DOT: Graph Description Language

Program Visualization

DCC

DOT: Graph Description Language

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
    a[i][j] = 0;
    }
  }
  for (i = 0; i < N; i++) {
    a[i][i] = 1;
  }
}
```

Program Visualization

DCC

DOT: Graph Description Language

```
$> clang -S -emit-llvm diag.c -o diag.ll
```

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization

DCC

DOT: Graph Description Language

```
$> clang -S -emit-llvm diag.c -o diag.ll
$> opt -dot-cfg diag.ll
```

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization

DCC

DOT: Graph Description Language

```
$> clang -S -emit-llvm diag.c -o diag.ll
$> opt -dot-cfg diag.ll
Writing '.identity.dot'...
```

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization

DCC

DOT: Graph Description Language

```
$> clang -S -emit-llvm diag.c -o diag.ll
$> opt -dot-cfg diag.ll
Writing '.identity.dot'...
$> cat .identity.dot
```

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization

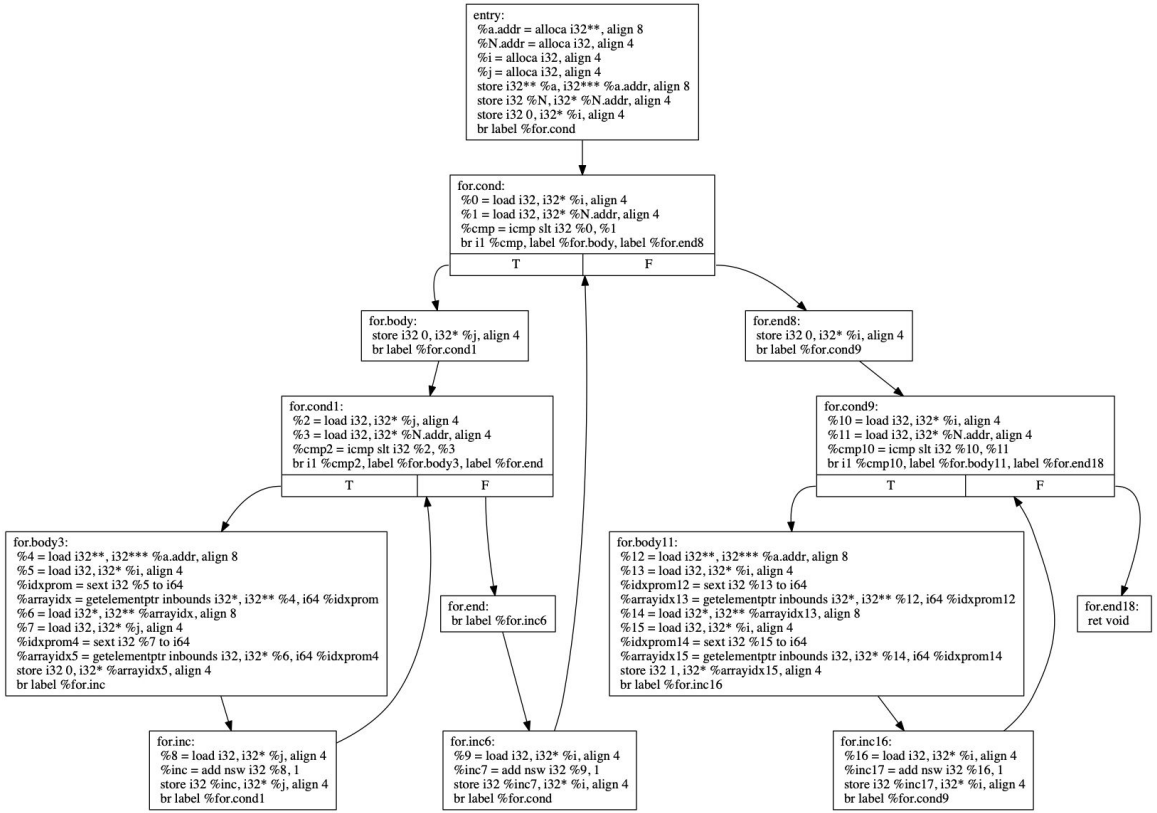
DCC

DOT: Graph Description Language

```
$> dot -Tpdf identity.dot -o identity.pdf
```

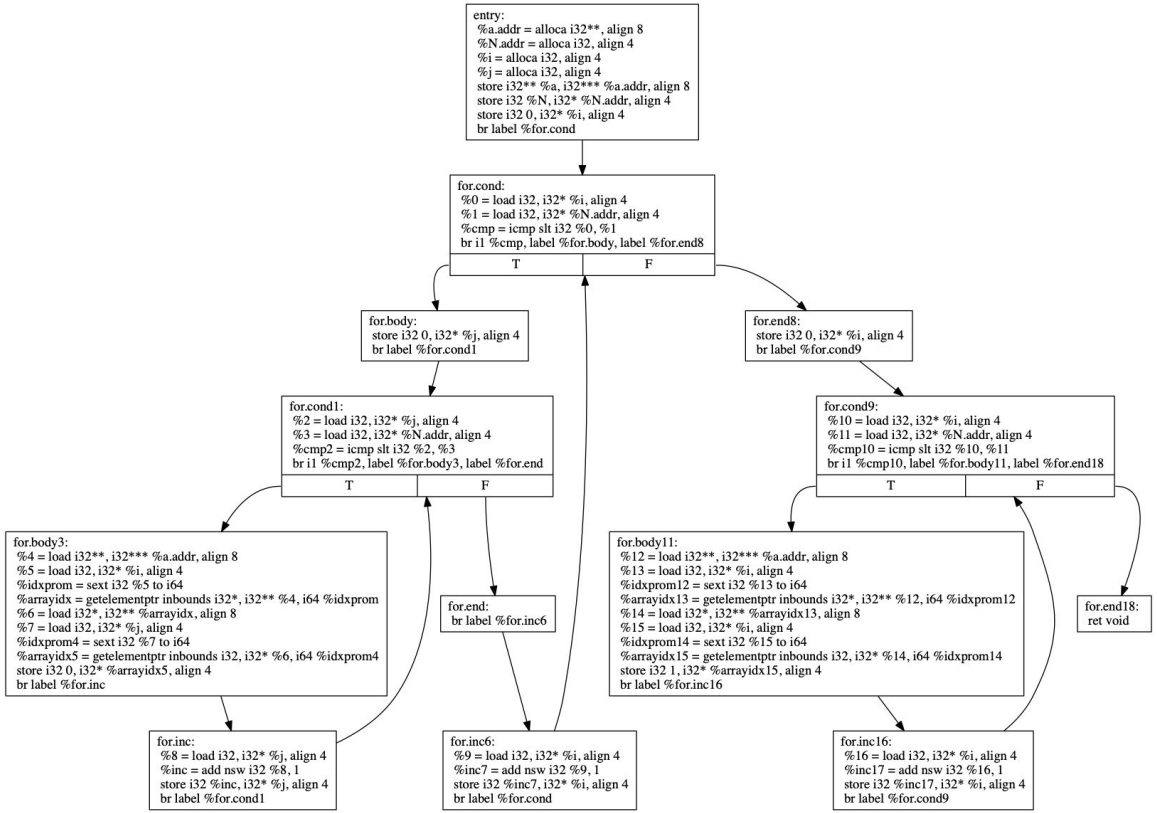
```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization




```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

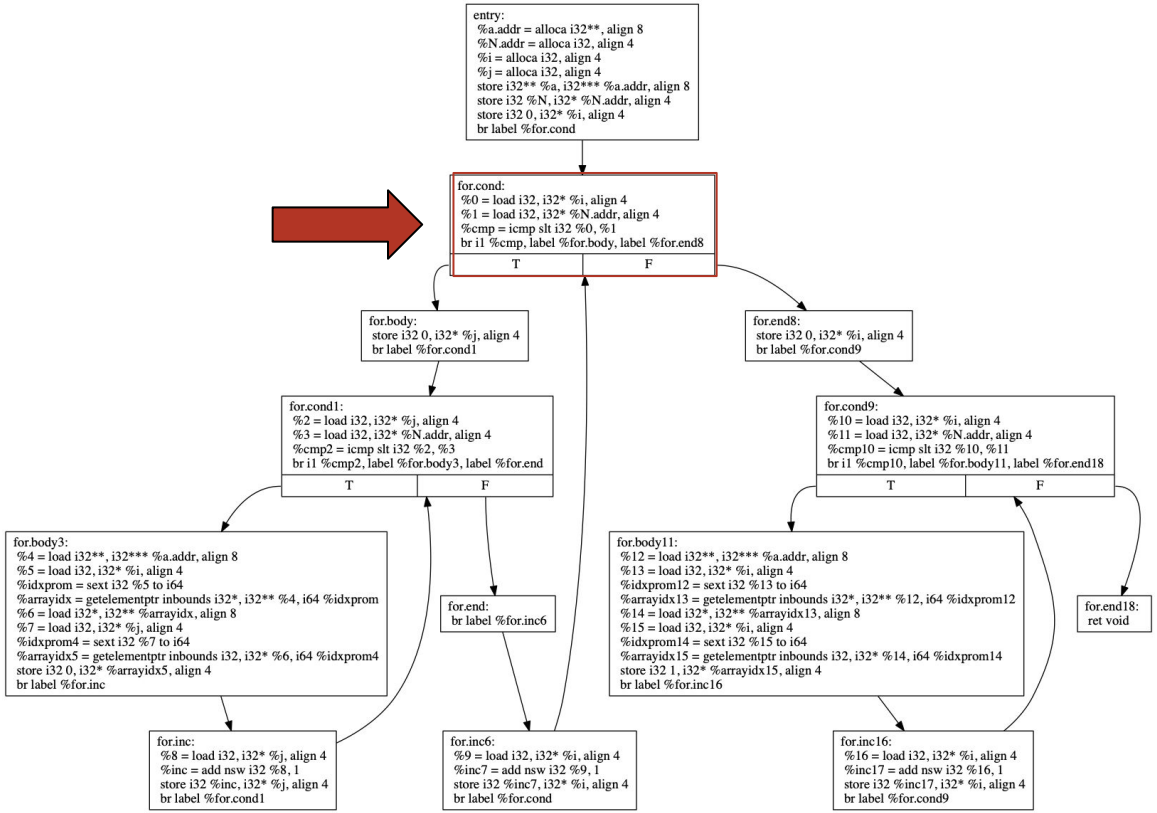
Program Visualization



Control Flow Graph

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization

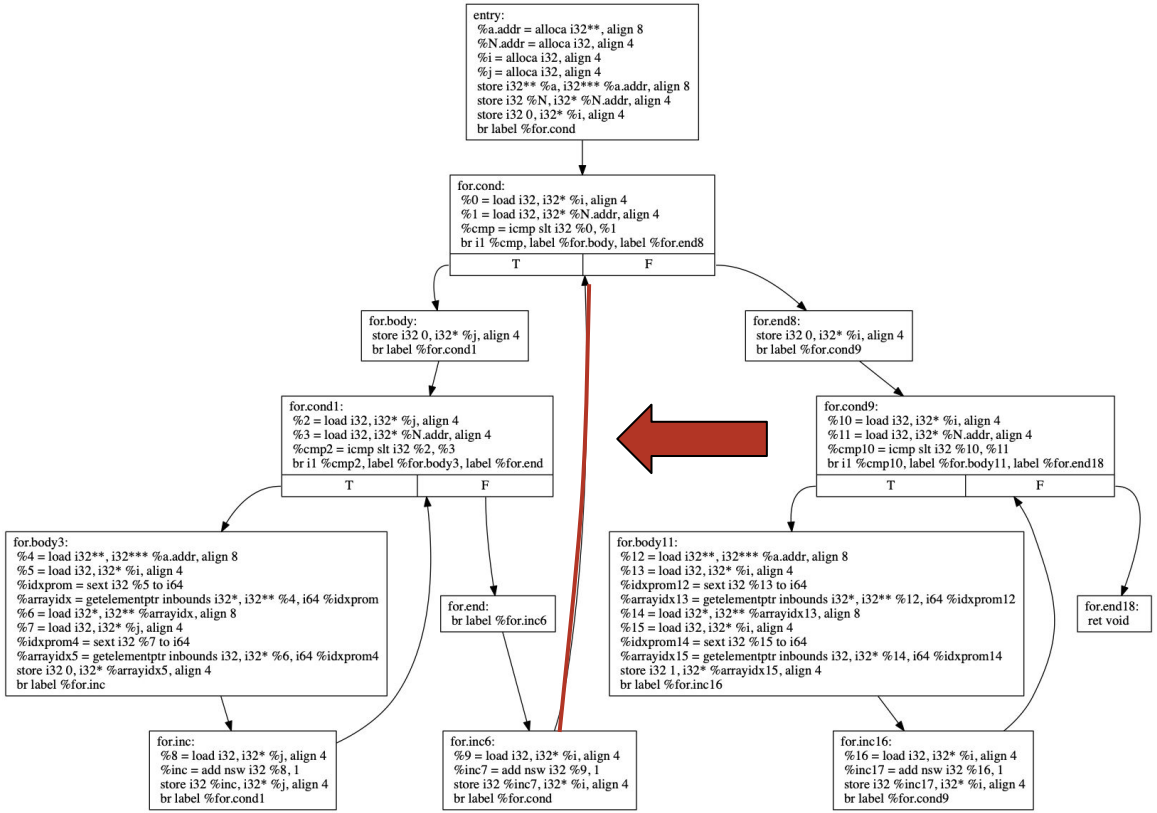


Control Flow Graph

- Vertex: Basic blocks

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization

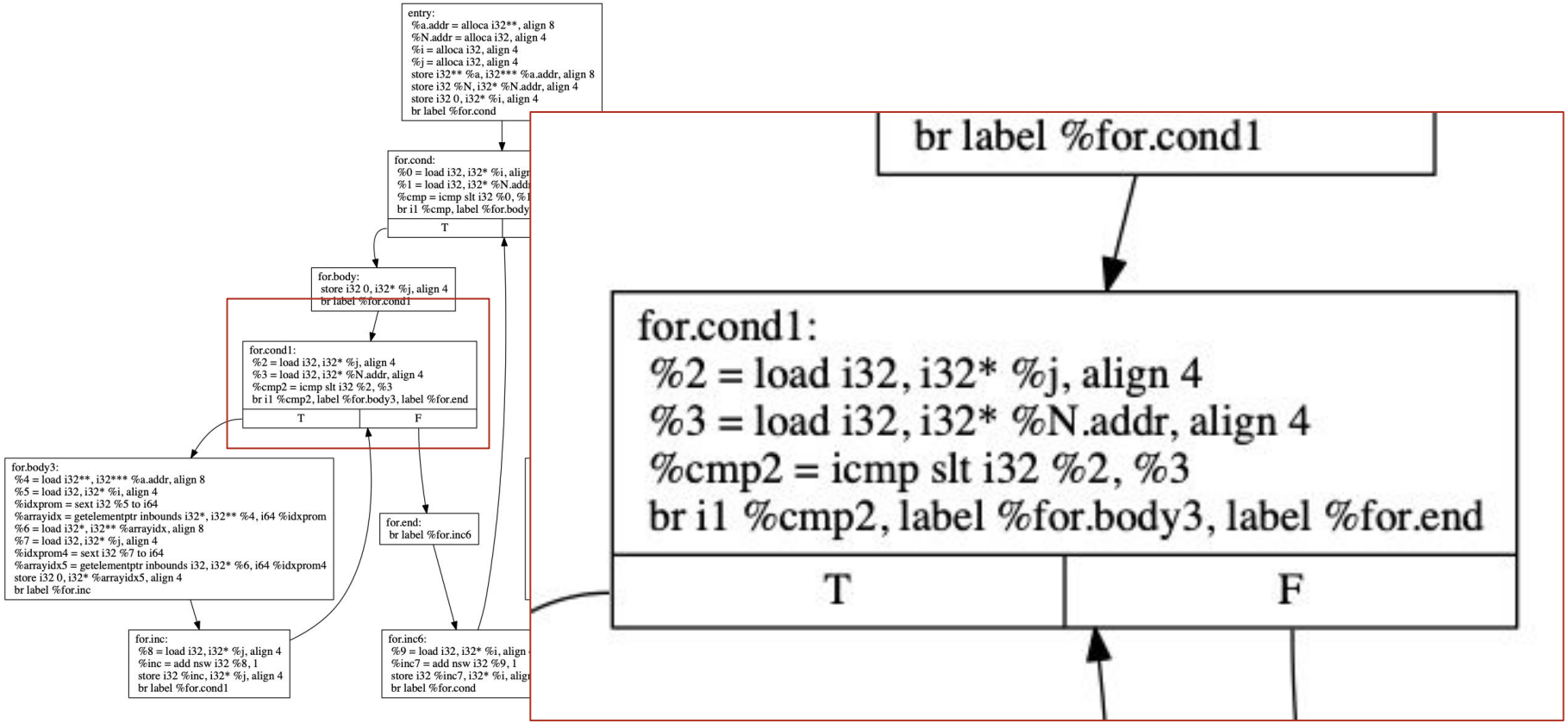


Control Flow Graph

- Vertex: Basic blocks
- Edge: Program flow

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Visualization



DOT Visualization

DOT Visualization

```
$> dot -Tpdf identity.dot -o identity.pdf
```

DOT Visualization

```
$> dot -Tpdf identity.dot -o identity.pdf
```

```
$> sudo apt install graphviz
```

DOT Visualization

DCC

dot

neato

twopi

circo

fdp

sfdp

```
$> dot -Tpdf identity.dot -o identity.pdf
$> sudo apt install graphviz
```

CFG Only

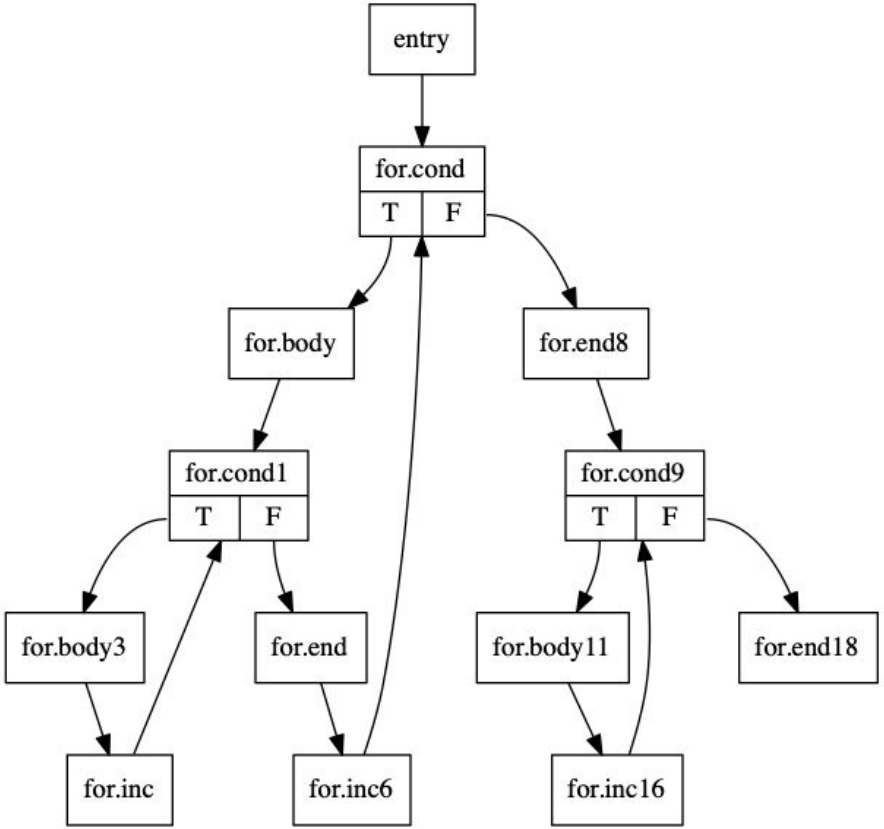
```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

CFG Only

```
$> opt -dot-cfg-only diag.ll
Writing '.identity.dot'...
```

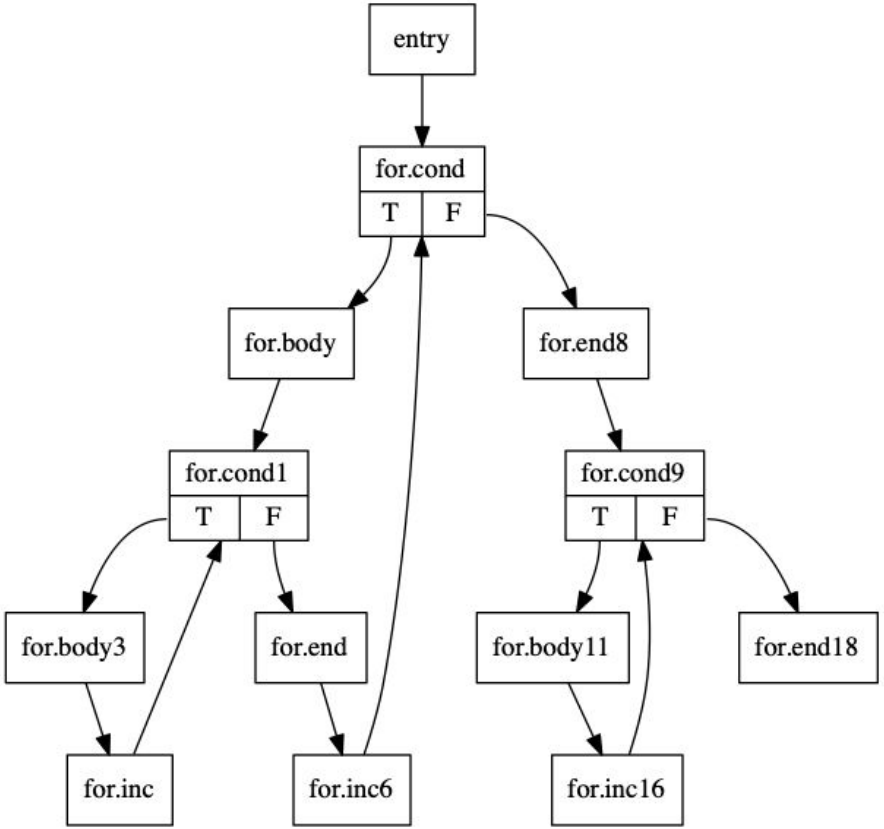
```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

CFG Only




```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

CFG Only




```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Regions

DCC

Program Regions

DCC

The Program Dependence Graph and Its Use in Optimization

JEANNE FERRANTE
IBM T. J. Watson Research Center
KARL J. OTTENSTEIN
Michigan Technological University
and
JOE D. WARREN
Rice University

Program Regions

DCC

The Program Dependence Graph and Its Use in Optimization

JEANNE FERRANTE
IBM T. J. Watson Research Center
KARL J. OTTENSTEIN
Michigan Technological University
and
JOE D. WARREN
Rice University

Program Regions

DCC

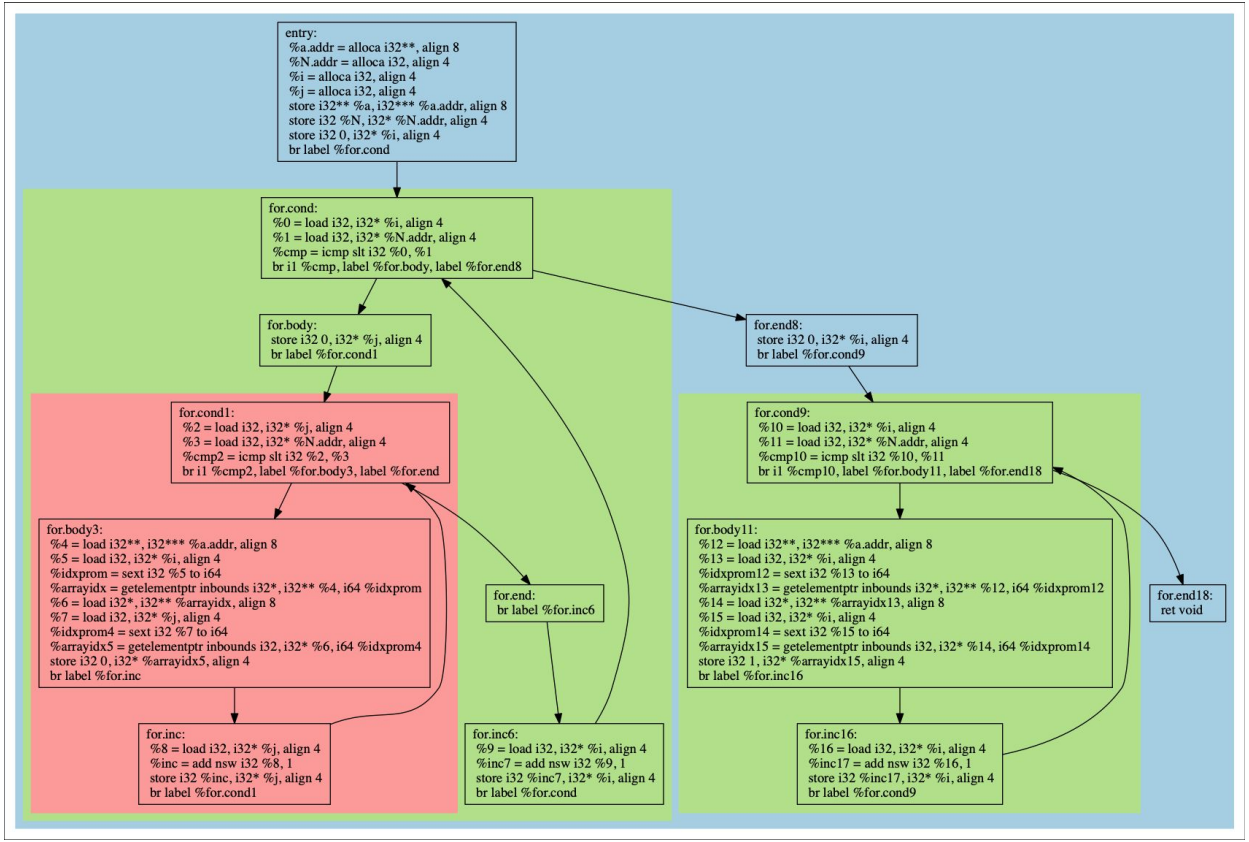
```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
     a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Regions

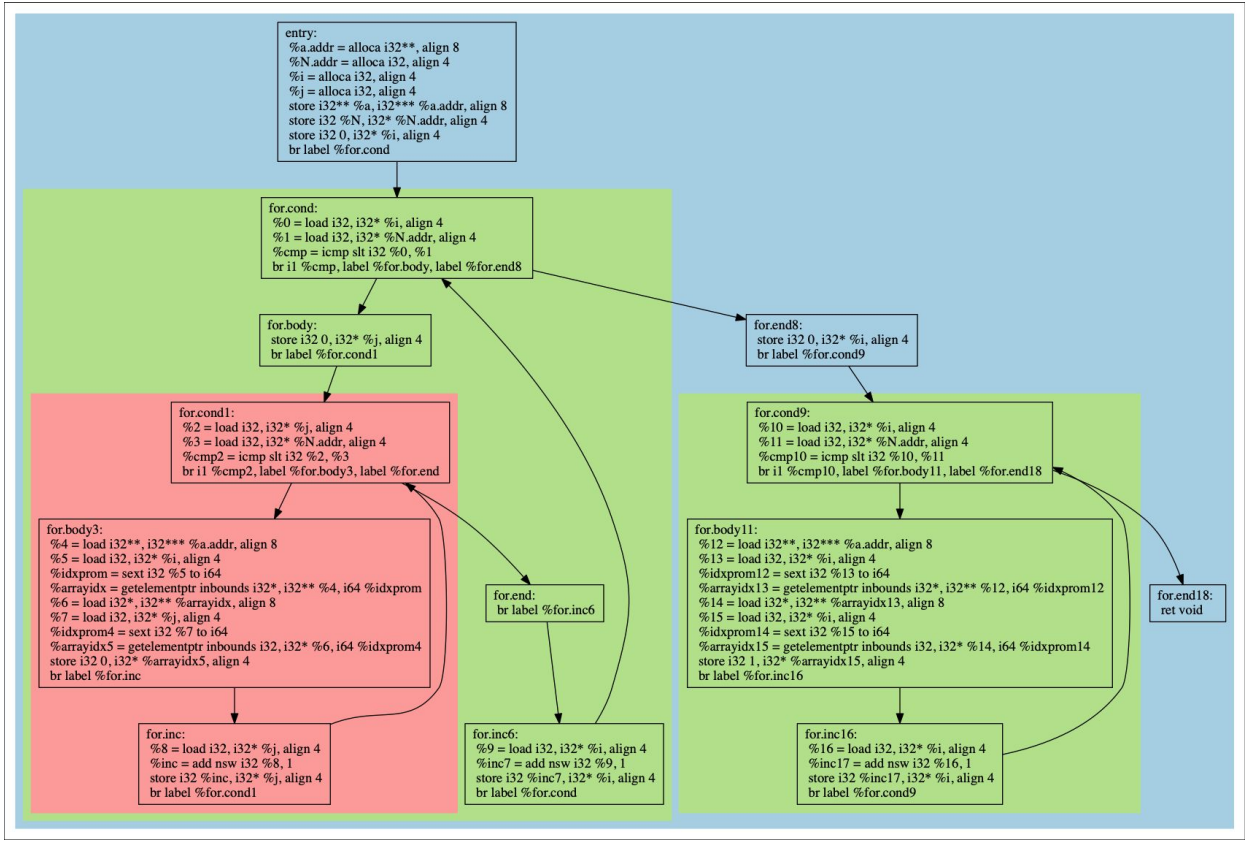
```
$> opt -dot-cfg-only diag.ll
Writing '.identity.dot'...
```

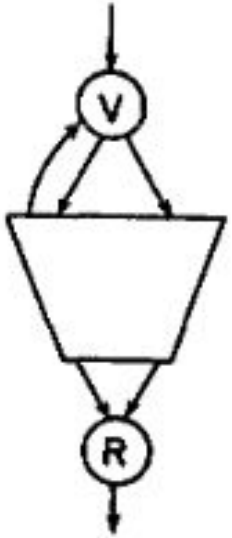
```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Program Regions



Program Regions





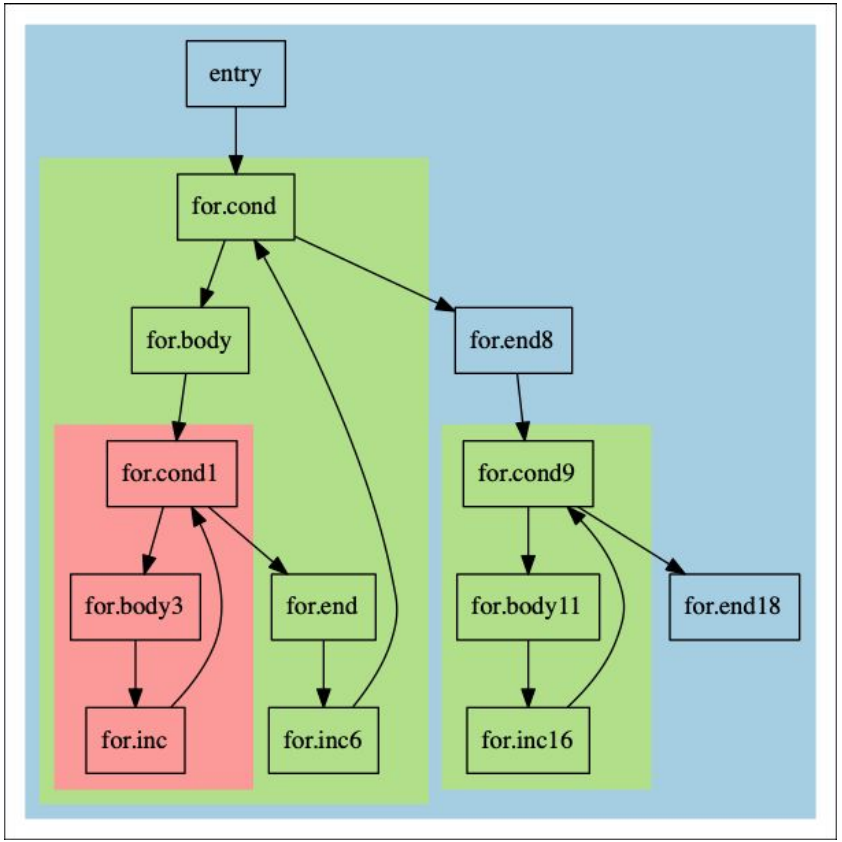
Ferrante'87

Regions Only

Regions Only

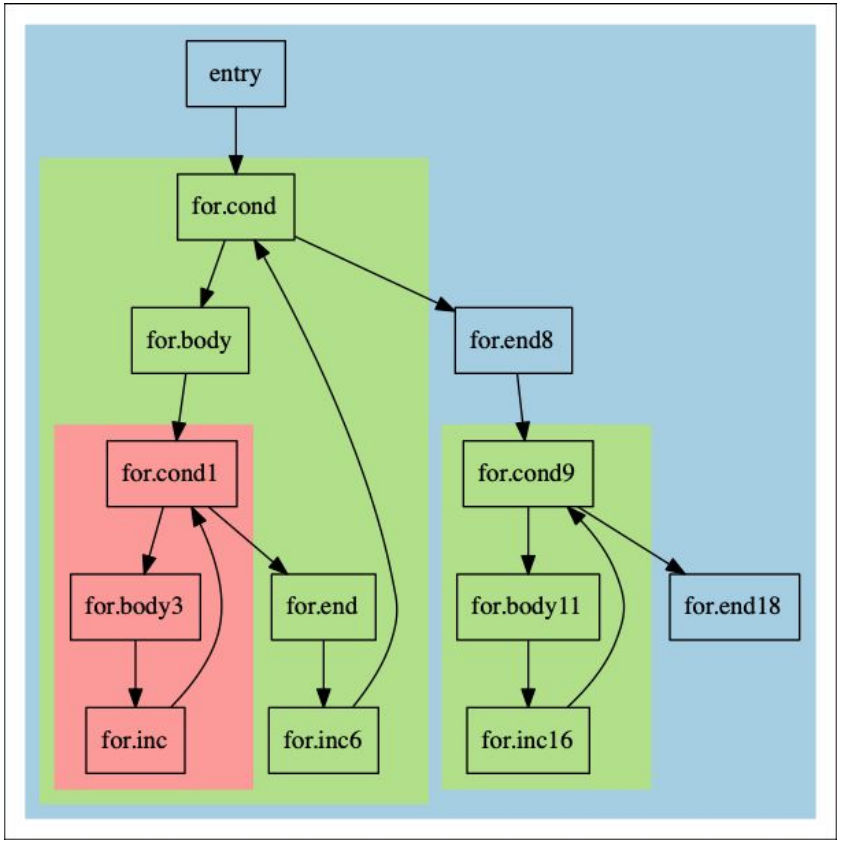
```
$> opt -dot-regions-only \
   diag.ll
```

Regions Only




```
$> opt -dot-regions-only \
   diag.ll
```

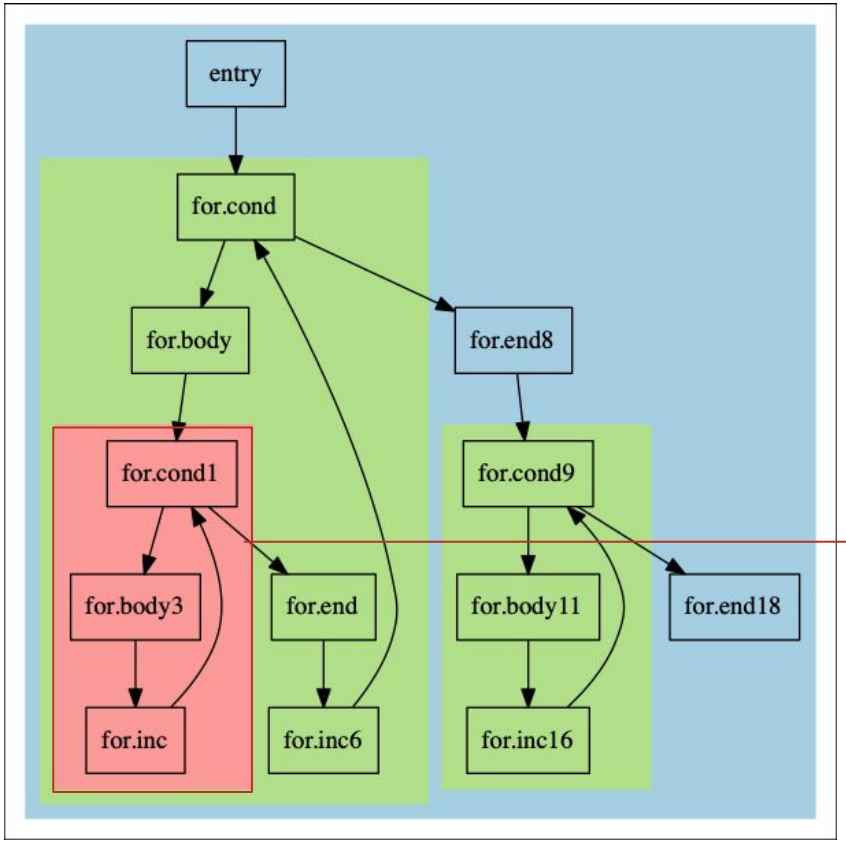
Regions Only




```
$> opt -dot-regions-only \
   diag.ll
```

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
      a[i][j] = 0;
    }
  }
  for (i = 0; i < N; i++) {
    a[i][i] = 1;
  }
}
```

Regions Only




```
$> opt -dot-regions-only \
   diag.ll
```

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
      a[i][j] = 0;
    }
  }
  for (i = 0; i < N; i++) {
    a[i][i] = 1;
  }
}
```

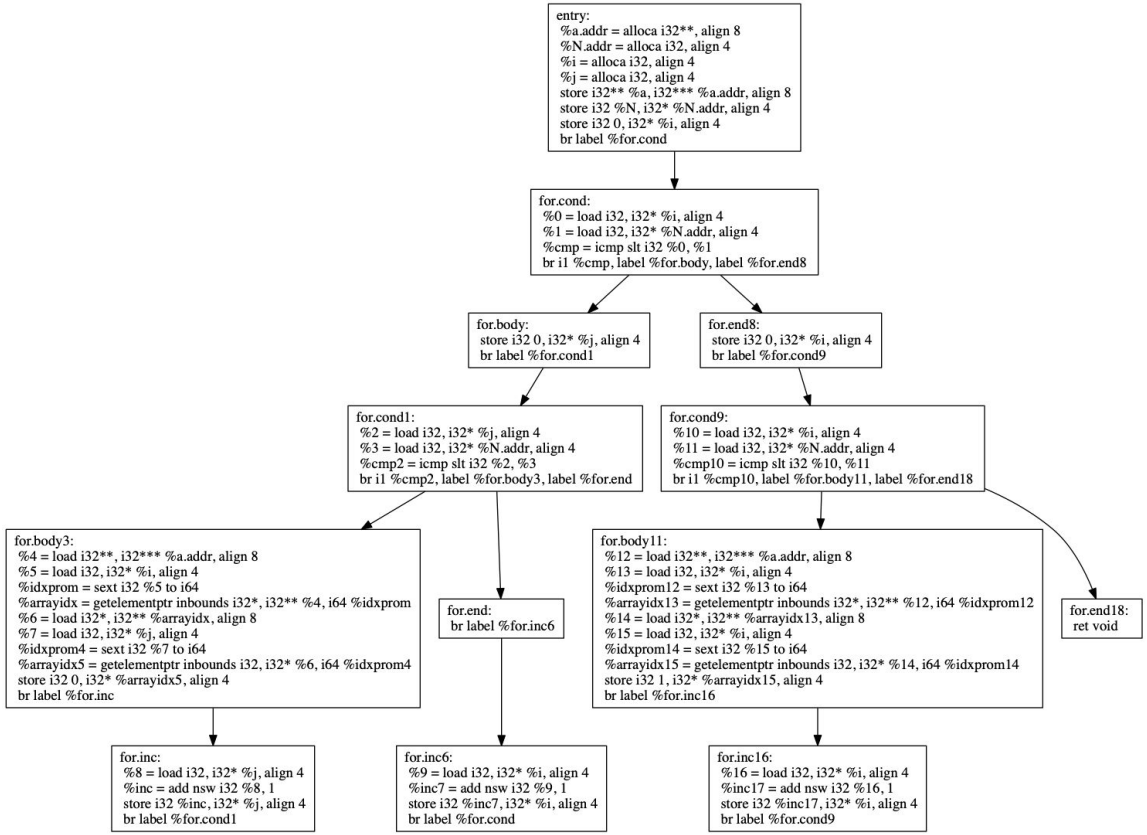
Dominator Tree

Dominator Tree

DCC

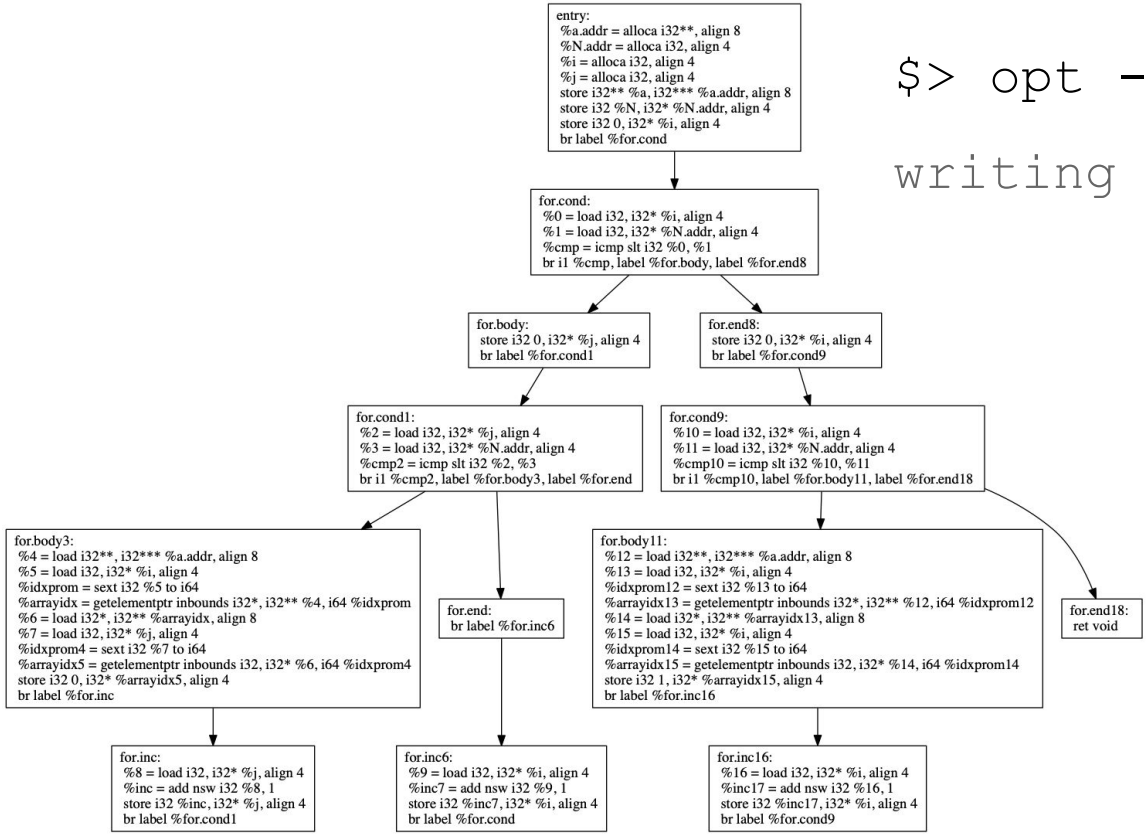
https://homepages.dcc.ufmg.br/~fernando/classes/dcc888/ementa/slides/LoopOptimizations.pdf

Dominator Tree




```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Dominator Tree




```
$> opt -dot-dom diag.ll
writing "dom.identity.dot"...
```

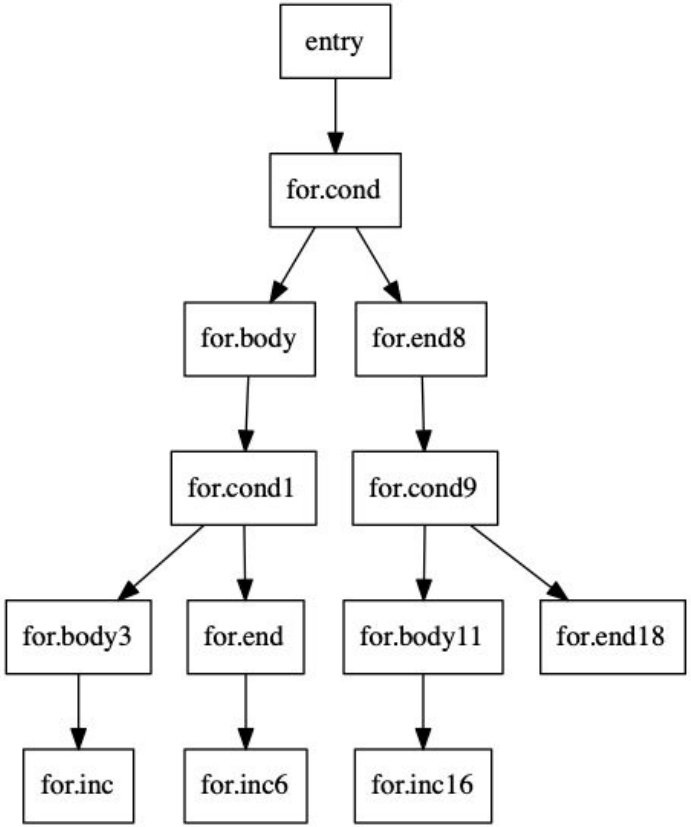
```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

Dominator Tree

```
$> opt -dot-dom-only diag.ll
writing
"domonly.identity.dot"...
```

```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

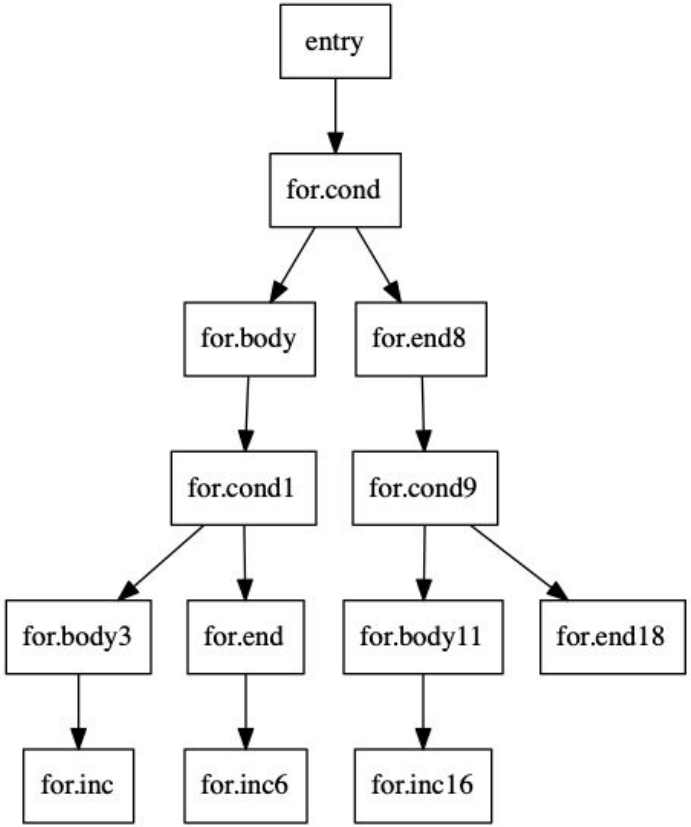
Dominator Tree

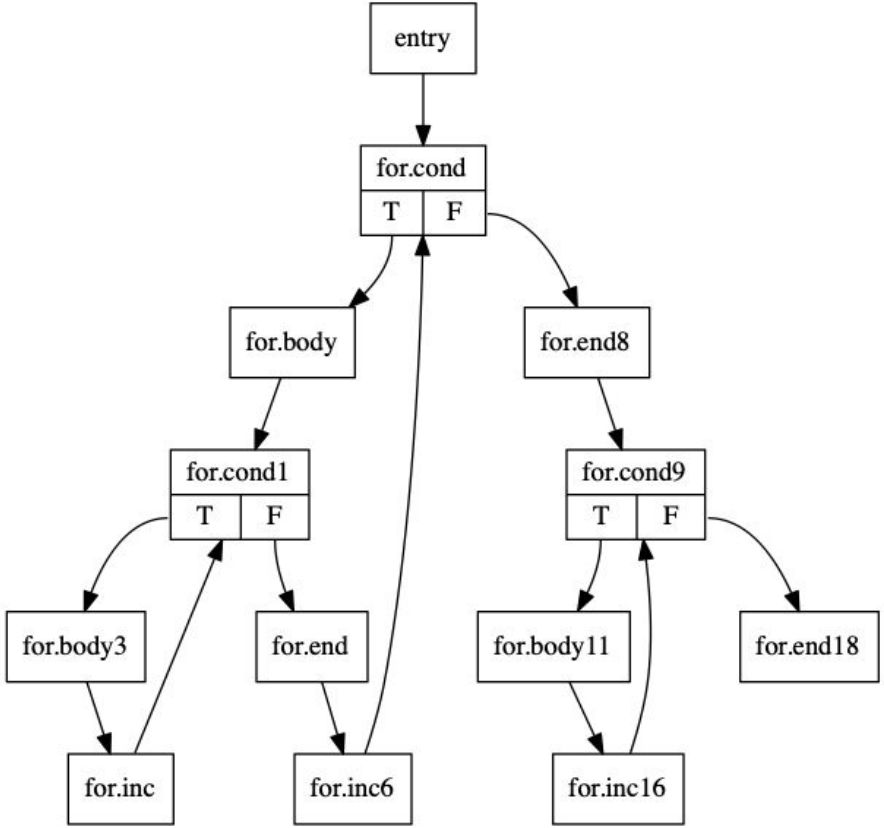



```
$> opt -dot-dom-only diag.ll
writing
"domonly.identity.dot"...
```

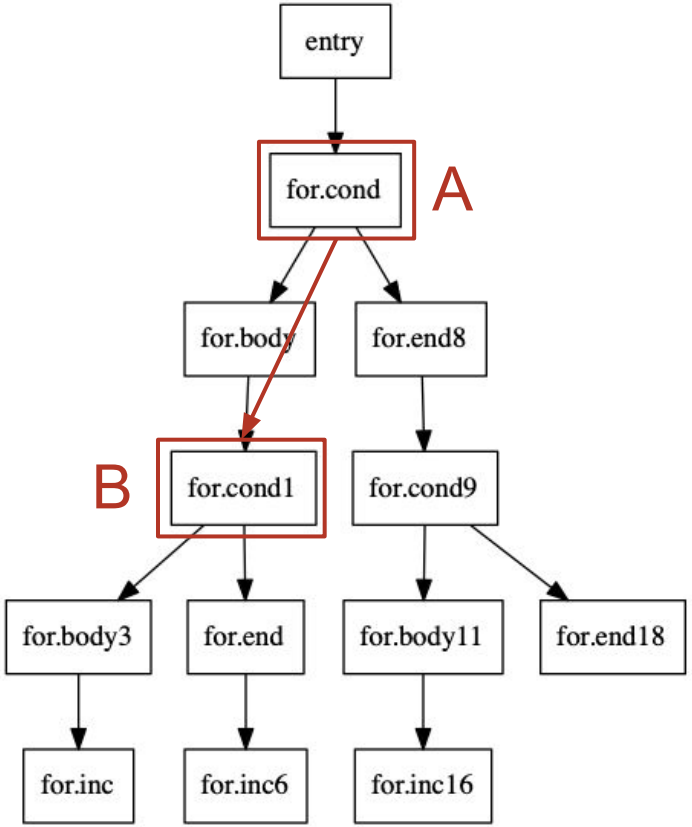
```
void identity(int** a, int N) {
  int i, j;
  for (i = 0; i < N; i++) {
     for (j = 0; j < N; j++) {
       a[i][j] = 0;
     }
  }
  for (i = 0; i < N; i++) {
     a[i][i] = 1;
  }
}
```

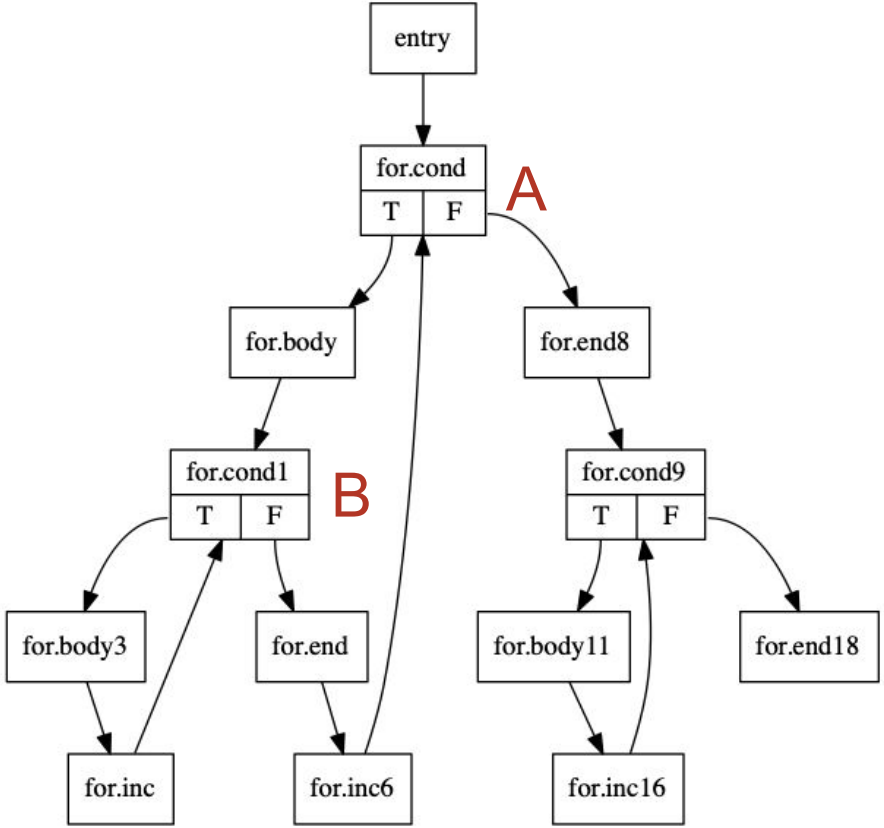
Dominator Tree





Dominator Tree





Many Formats

```
$> opt --help | grep dot
```

Many Formats

```
$> opt --help | grep dot
    --dot-callgraph
    --dot-cfg
    --dot-cfg-only
    --dot-dom
    --dot-dom-only
    --dot-postdom
    --dot-postdom-only
    --dot-regions
    --dot-regions-only
```

Many Formats

```
$> opt --help | grep dot
    --dot-callgraph
    --dot-cfg
    --dot-cfg-only
    --dot-dom
    --dot-dom-only
    --dot-postdom
    --dot-postdom-only
    --dot-regions
    --dot-regions-only
```

Many Formats

```
$> opt --help | grep dot
    --dot-callgraph
    --dot-cfg
    --dot-cfg-only
    --dot-dom
    --dot-dom-only
    --dot-postdom
    --dot-postdom-only
    --dot-regions
    --dot-regions-only
```

Many Formats

```
$> opt --help | grep dot
    --dot-callgraph
    --dot-cfg
    --dot-cfg-only
    --dot-dom
    --dot-dom-only
    --dot-postdom
    --dot-postdom-only
```

https://homepages.dcc.ufmg.br/~fernando/classes/dcc888/ementa/slides/SparseAbstractInterpretation.pdf

Many Formats

```
$> opt --help | grep dot
    --dot-callgraph
    --dot-cfg
    --dot-cfg-only
    --dot-dom
    --dot-dom-only
    --dot-postdom
    --dot-postdom-only
    --dot-regions
    --dot-regions-only
```

The Call Graphs

The Call Graphs

DCC

```
int fact(int n) {
  int ans = 1;
  while (n > 1) {
    ans *= n;
    n--;
  }
  return ans;
}

int main() {
  return fact(3);
}
```

The Call Graphs

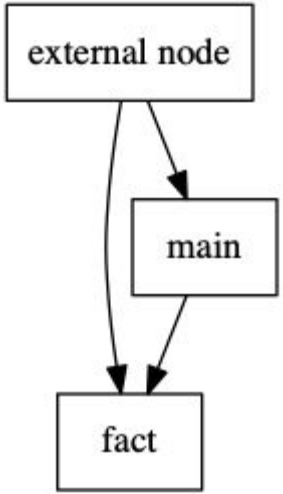
```
$> opt -dot-callgraph fact.ll
writing "callgraph.dot"...
```

```
int fact(int n) {
  int ans = 1;
  while (n > 1) {
    ans *= n;
    n--;
  }
  return ans;
}

int main() {
  return fact(3);
}
```

The Call Graphs

```
$> opt -dot-callgraph fact.ll
writing "callgraph.dot"...
```

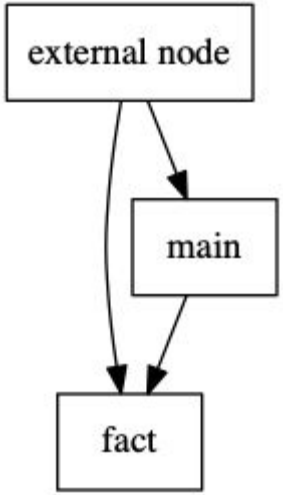



```
int fact(int n) {
  int ans = 1;
  while (n > 1) {
    ans *= n;
    n--;
  }
  return ans;
}

int main() {
  return fact(3);
}
```

The Call Graphs

```
$> opt -dot-callgraph fact.ll
writing "callgraph.dot"...
```

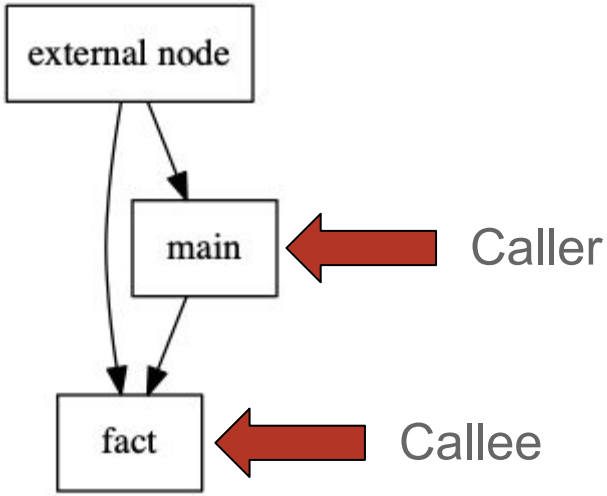



```
int fact(int n) {
  int ans = 1;
  while (n > 1) {
    ans *= n;
    n--;
  }
  return ans;
}

int main() {
  return fact(3);
}
```

The Call Graphs

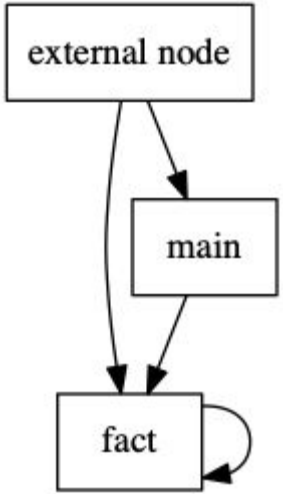
```
$> opt -dot-callgraph fact.ll
writing "callgraph.dot"...
```




```
int fact(int n) {
  int ans = 1;
  while (n > 1) {
    ans *= n;
    n--;
  }
  return ans;
}

int main() {
  return fact(3);
}
```

The Call Graphs




```
int fact(int n) {
  return n > 1 ? n * fact(n-1) : 1;
}

int main() {
  return fact(3);
}
```

Contact & References

DCC

Fernando Magno Quintão Pereira

fernando@dcc.ufmg.br

http://lac.dcc.ufmg.br

References

https://llvm.org/docs/LangRef.html